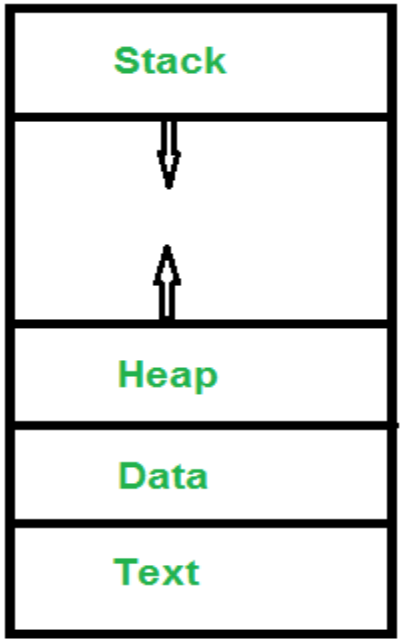
PROCESS MANAGEMENT

Process

A process is a program in execution. For example, when we write a program in C or C++ and compile it, the compiler creates binary code. The original code and binary code are both programs. When we actually run the binary code, it becomes a process.

A process is an ‘active’ entity instead of a program, which is considered a ‘passive’ entity. A single program can create many processes when run multiple times; for example, when we open a .exe or binary file multiple times, multiple instances begin (multiple processes are created).



What does a process look like in memory?

Stack

The process Stack contains the temporary data such as method/ return address and local variables.

Heap

This is dynamically allocated memory to a process during its run time.

Text

This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.

Data

This section contains the global and static variables.

CPU-Bound vs I/O-Bound Processes:

A CPU-bound process requires more CPU time or spends more time in the running state.

An I/O-bound process requires more I/O time and less CPU time. An I/O-bound process spends more time in the waiting state.

Operations on the Process

1. **Creation**

Once the process is created, it will be ready and come into the ready queue (main memory) and will be ready for the execution.

2**. Scheduling**

Out of the many processes present in the ready queue, the Operating system chooses one process and start executing it. Selecting the process which is to be executed next, is known as scheduling.

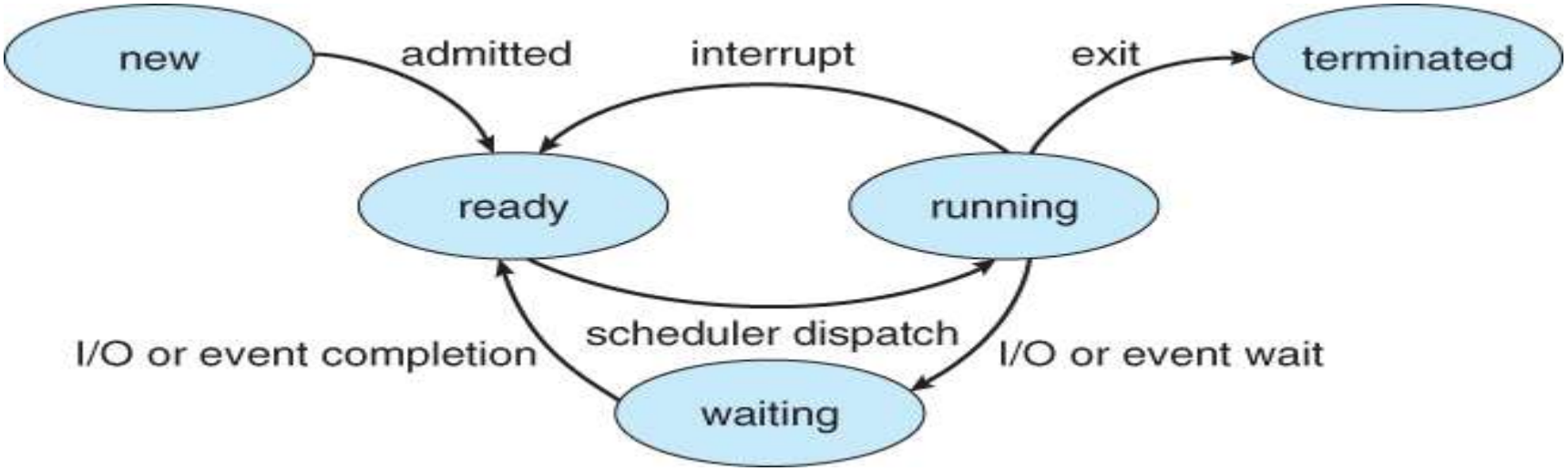
3. **Execution**

Once the process is scheduled for the execution, the processor starts executing it. Process may come to the blocked or wait state during the execution then in that case the processor starts executing the other processes.

4**. Deletion/killing**

Once the purpose of the process gets over then the OS will kill the process. The Context of the process (PCB) will be deleted and the process gets terminated by the Operating system.

Process Life Cycle



- **New/Start**

This is the initial state when a process is first started/created.

- **Ready**

The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. Process may come into this state after **Start** state or while running it by but interrupted by the scheduler to assign CPU to some other process.

- **Running**

Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.

- **Waiting**

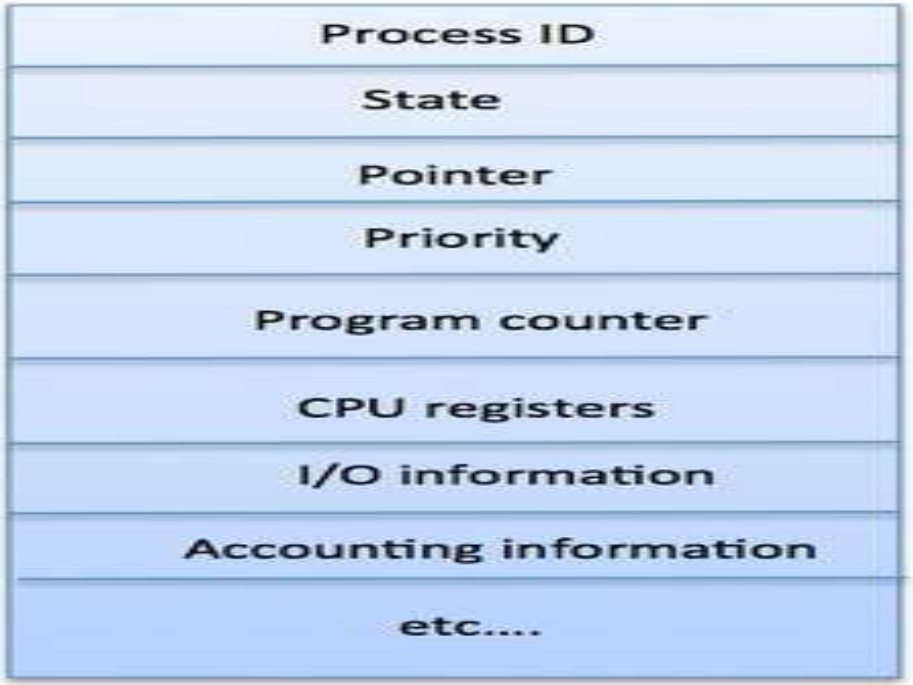
Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.

- **Terminated or Exit**

Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.

Process Control Block (PCB)

- A Process Control Block is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID). A PCB keeps all the information needed to keep track of a process.
- The architecture of a PCB is completely dependent on Operating System and may contain different information in different operating systems. Here is a simplified diagram of a PCB.
- The PCB is maintained for a a process throughout its lifetime, and is deleted once the process terminates.



1. **Process ID**

When a process is created, a unique id is assigned to the process which is used for unique identification of the process in the system.

2. Program counter

A program counter stores the address of the last instruction of the process on which the process was suspended. The CPU uses this address when the execution of this process is resumed.

3. Process State

The Process, from its creation to the completion, goes through various states which are new, ready, running and waiting.

4. Priority

Every process has its own priority. The process with the highest priority among the processes gets the CPU first. This is also stored on the process control block.

5. General Purpose Registers

Every process has its own set of registers which are used to hold the data which is generated during the execution of the process.

6. List of open files

During the Execution, Every process uses some files which need to be present in the main memory. OS also maintains a list of open files in the PCB.

7. List of open devices

OS also maintain the list of all open devices which are used during the execution of the process.

Context Switching:

Context Switching involves storing the context or state of a process so that it can be reloaded when required and execution can be resumed from the same point as earlier. This is a feature of a multitasking operating system and allows a single CPU to be shared by multiple processes.

When does context switching happen?

There are three major triggers for context switching. These are given as follows −

- **Multitasking:** In a multitasking environment, a process is switched out of the CPU so another process can be run. The state of the old process is saved and the state of the new process is loaded. On a pre-emptive system, processes may be switched out by the scheduler.
- **Interrupt Handling:** The hardware switches a part of the context when an interrupt occurs. This happens automatically. Only some of the context is changed to minimize the time required to handle the interrupt.
- **User and Kernel Mode Switching:** A context switch may take place when a transition between the user mode and kernel mode is required in the operating system.

Context Switching Steps

The steps involved in context switching are as follows −

- Save the context of the process that is currently running on the CPU. Update the process control block and other important fields.
- Move the process control block of the above process into the relevant queue such as the ready queue, I/O queue etc.
- Select a new process for execution.
- Update the process control block of the selected process. This includes updating the process state to running.
- Update the memory management data structures as required.
- Restore the context of the process that was previously running when it is loaded again on the processor. This is done by loading the previous values of the process control block and registers.

Inter Process Communication (IPC)

A process can be of two types:

- Independent process.
- Co-operating process.

An independent process is not affected by the execution of other processes while a co-operating process can be affected by other executing processes.

Inter-process communication (IPC) is a mechanism that allows processes to communicate with each other and synchronize their actions.

The communication between these processes can be seen as a method of co-operation between them. Processes can communicate with each other through both:

- Shared Memory
- Message passing

Approaches for Inter-Process Communication



- **Pipes**

Pipe is widely used for communication between two related processes. This is a half-duplex method, so the first process communicates with the second process. However, in order to achieve a full-duplex, another pipe is needed. These pipes are used in all types of POSIX systems and in different versions of window operating systems as well.

- **Message Passing:**

It is a mechanism for a process to communicate and synchronize. Using message passing, the process communicates with each other without resorting to shared variables.

IPC mechanism provides two operations:

- Send (message)- message size fixed or variable
- Received (message)

- **Message Queues:**

A message queue is a linked list of messages stored within the kernel. It is identified by a message queue identifier. This method offers communication between single or multiple processes with full-duplex capacity.

- **Direct Communication:**

In this type of inter-process communication process, should name each other explicitly. In this method, a link is established between one pair of communicating processes, and between each pair, only one link exists.

- **Indirect Communication:**

Indirect communication establishes like only when processes share a common mailbox each pair of processes sharing several communication links. A link can communicate with many processes. The link may be bi-directional or unidirectional.

- **Shared Memory:**

Shared memory is a memory shared between two or more processes that are established using shared memory between all the processes. This type of memory requires to protected from each other by synchronizing access across all the processes.

- **FIFO:**

Communication between two unrelated processes. It is a full-duplex method, which means that the first process can communicate with the second process, and the opposite can also happen.

Why we need interprocess communication?

There are numerous reasons to use inter-process communication for sharing the data. Some of the most important reasons are given below:

- It helps to speedup modularity
- Computational
- Privilege separation
- Convenience
- Helps operating system to communicate with each other and synchronize their actions as well.

IPC through shared memory

Inter Process Communication through shared memory is a concept where two or more process can access the common memory. And communication is done via this shared memory where changes made by one process can be viewed by another process.

The problem with pipes, fifo and message queue – is that for two process to exchange information. The information has to go through the kernel.

- Server reads from the input file.
- The server writes this data in a message using either a pipe, fifo or message queue.
- The client reads the data from the IPC channel, again requiring the data to be copied from kernel's IPC buffer to the client's buffer.
- Finally the data is copied from the client's buffer.

A total of four copies of data are required (2 read and 2 write). So, shared memory provides a way by letting two or more processes share a memory segment. With Shared Memory the data is only copied twice – from input file into shared memory and from shared memory to the output file.

Message Passing Model of Process Communication

Message passing means how a message can be sent from one end to the other end. Either it may be a client-server model or it may be from one node to another node. The formal model for distributed message passing has two timing models one is synchronous and the other is asynchronous.

The fundamental points of message passing are:

- In message-passing systems, processors communicate with one another by sending and receiving messages over a communication channel. So how the arrangement should be done?
- The pattern of the connection provided by the channel is described by some topology systems.
- The collection of the channels are called a network.
- So by the definition of distributed systems, we know that they are geographically set of computers. So it is not possible for one computer to directly connect with some other node.

- So all channels in the Message-Passing Model are private.
- The sender decides what data has to be sent over the network. An example is, making a phone call.
- The data is only fully communicated after the destination worker decides to receive the data. Example when another person receives your call and starts to reply to you.
- There is no time barrier. It is in the hand of a receiver after how many rings he receives your call. He can make you wait forever by not picking up the call.
- For successful network communication, it needs active participation from both sides.

Shared Memory Model	Message Passing Model
Shared memory region is used for communication.	Message passing facility is used for communication.
It is used for communication between processes on a single processor or multiprocessor systems where the communicating processes reside on the same machine as the communicating processes share a common address space.	It is typically used in a distributed environment where communicating processes reside on remote machines connected through a network.
The code for reading and writing the data from the shared memory should be written explicitly by the Application programmer.	No such code required here as the message passing facility provides mechanism for communication and synchronization of actions performed by the communicating processes.
It provides maximum speed of computation as communication is done through shared memory so system calls are made only to establish the shared memory.	It is time consuming as message passing is implemented through kernel intervention (system calls).
Here the processes need to ensure that they are not writing to the same location simultaneously.	It is useful for sharing small amounts of data as conflicts need not to be resolved.
Faster communication strategy.	Relatively slower communication strategy

Process Schedulers

There are three types of process scheduler.

- **Long Term or job scheduler :**

It brings the new process to the 'Ready State'. It controls ***Degree of Multi-programming***, i.e., number of process present in ready state at any point of time. It is important that the long-term scheduler make a careful selection of both I/O and CPU-bound processes. I/O bound tasks are which use much of their time in input and output operations while CPU bound processes are which spend their time on CPU. The job scheduler increases efficiency by maintaining a balance between the two.

- **Short term or CPU scheduler :**

It is responsible for selecting one process from ready state for scheduling it on the running state. Note: Short-term scheduler only selects the process to schedule it doesn't load the process on running. Here is when all the scheduling algorithms are used. The CPU scheduler is responsible for ensuring there is no starvation owing to high burst time processes.

Dispatcher is responsible for loading the process selected by Short-term scheduler on the CPU (Ready to Running State) Context switching is done by dispatcher only. A dispatcher does the following:

–Switching context.

–Switching to user mode.

–Jumping to the proper location in the newly loaded program.

- **Medium-term scheduler :**

It is responsible for suspending and resuming the process. It mainly does swapping (moving processes from main memory to disk and vice versa). Swapping may be necessary to improve the process mix or because a change in memory requirements has overcommitted available memory, requiring memory to be freed up. It is helpful in maintaining a perfect balance between the I/O bound and the CPU bound. It reduces the degree of multiprogramming.

PRE-EMPTIVE &NON PRE-EMPTIVE SCHEDULING

Preemptive Scheduling:

Preemptive scheduling is used when a process switches from running state to ready state or from the waiting state to ready state. The resources (mainly CPU cycles) are allocated to the process for a limited amount of time and then taken away, and the process is again placed back in the ready queue if that process still has CPU burst time remaining. That process stays in the ready queue till it gets its next chance to execute.

Non-Preemptive Scheduling:

Non-preemptive Scheduling is used when a process terminates, or a process switches from running to the waiting state. In this scheduling, once the resources (CPU cycles) are allocated to a process, the process holds the CPU till it gets terminated or reaches a waiting state. In the case of non-preemptive scheduling does not interrupt a process running CPU in the middle of the execution. Instead, it waits till the process completes its CPU burst time, and then it can allocate the CPU to another process.

PRE-EMPTIVE vs NON PRE-EMPTIVE SCHEDULING

- Preemptive scheduling is whereby the processes with higher priorities are executed first. A process can be interrupted by another process in the middle of its execution. On the other hand, non-preemptive scheduling is whereby once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.
- Non-preemptive scheduling can be described as rigid because even if a critical process enters the ready queue, the process running CPU is not interrupted. On the other hand, preemptive scheduling can be described as flexible because it allows the critical processes to access CPU as they arrive into the ready queue no matter what process is currently being executed.

- In preemptive scheduling, if a process with higher priority enters in a ready queue, the process with lower priority is removed while in non-preemptive scheduling once the process has been allocated to the CPU, it will complete its execution.
- In non-preemptive scheduling, the process cannot be scheduled whereas in the preemptive scheduling, the process can be scheduled.
- Preemptive scheduling can cause a problem when two processes share data because one may be interrupted in the middle of updating shared data structures. This is usually not the case with non-preemptive scheduling.
- Non-preemptive scheduling has no overhead of switching the process from running state to ready state whereas preemptive scheduling, has the overhead of switching the process from ready state to running state, running state to ready state.
- In preemptive scheduling, the CPU utilization is higher than in non-preemptive scheduling.

- In the non-preemptive scheduling, if the CPU is allocated to the process having larger burst time then the processes with small burst time may have to starve. In contrast, in preemptive scheduling, if a high priority process frequently arrives in the ready queue then the process with low priority has to wait for long, and it may have to starve.
- The preemptive scheduling is costly when compared to non-preemptive scheduling because it has to maintain the integrity of shared data.

Times associated with Processes in CPU Scheduling algorithms

- **Arrival Time:** Time at which the process arrives in the ready queue.
- **Completion Time:** Time at which process completes its execution.
- **Burst Time:** Time required by a process for CPU execution.
- **Turn Around Time:** The total amount of time spent by the process from its arrival to its completion, is called Turnaround time.

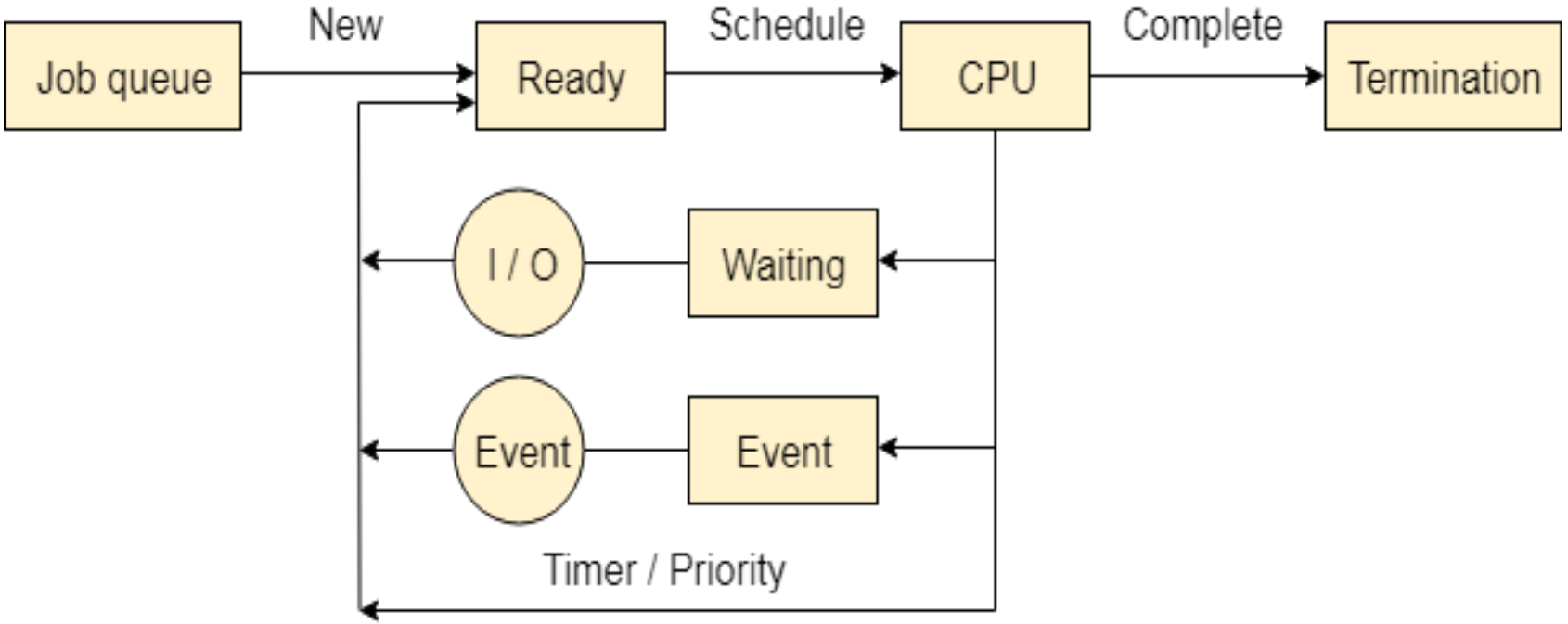
  Turn Around Time = Completion Time – Arrival Time
- **Waiting Time(W.T):** The Total amount of time for which the process waits for the CPU to be assigned is called waiting time.

  Waiting Time = Turn Around Time – Burst Time **Or**

  Waiting Time = Start time - Arrival time
- **Response Time :** The difference between the arrival time and the time at which the process first gets the CPU is called Response Time.

Process Queues

The Operating system manages various types of queues for each of the process states. The PCB related to the process is also stored in the queue of the same state. If the Process is moved from one state to another state then its PCB is also unlinked from the corresponding queue and added to the other state queue in which the transition is made.



- **Job Queue**

In starting, all the processes get stored in the job queue. It is maintained in the secondary memory. The long term scheduler (Job scheduler) picks some of the jobs and put them in the primary memory.

- **Ready Queue**

Ready queue is maintained in primary memory. The short term scheduler picks the job from the ready queue and dispatch to the CPU for the execution.

- **Waiting Queue**

When the process needs some IO operation in order to complete its execution, OS changes the state of the process from running to waiting. The context (PCB) associated with the process gets stored on the waiting queue which will be used by the Processor when the process finishes the IO.

SCHEDULING CRITERIA

Different CPU scheduling algorithms have different properties and the choice of a particular algorithm depends on the various factors. Many criteria have been suggested for comparing CPU scheduling algorithms.

The criteria include the following:

CPU utilization

The main purpose of any CPU algorithm is to keep the CPU as busy as possible. Theoretically, CPU usage can range from 0 to 100 but in a real-time system, it varies from 40 to 90 percent depending on the system load.

Throughput –

The average CPU performance is the number of processes performed and completed during each unit. This is called throughput. The output may vary depending on the length or duration of the processes.

Turnaround time

The time elapsed from the time of submission of a process to the time of completion is known as the turnaround time. Turn-around time is the sum of times spent waiting to get into memory, waiting in ready queue, executing in CPU, and waiting for I/O.

Waiting time

The sum of the periods spent waiting in the ready queue amount of time a process has been waiting in the ready queue to acquire get control on the CPU.

Response time –

Amount of time it takes from when a request was submitted until the first response is produced. Remember, it is the time till the first response and not the completion of process execution(final response).

In general CPU utilization and Throughput are maximized and other factors are reduced for proper optimization.

SCHEDULING ALGORITHMS

There are various algorithms which are used by the Operating System to schedule the processes on the processor in an efficient way.

The Purpose of a Scheduling algorithm:

- Maximum CPU utilization
- Fare allocation of CPU
- Maximum throughput
- Minimum turnaround time
- Minimum waiting time
- Minimum response time

- **First Come First Serve**

It is the simplest algorithm to implement. The process with the minimal arrival time will get the CPU first. The lesser the arrival time, the sooner will the process gets the CPU. It is the non-preemptive type of scheduling.

- **Round Robin**

In the Round Robin scheduling algorithm, the OS defines a time quantum (slice). All the processes will get executed in the cyclic way. Each of the process will get the CPU for a small amount of time (called time quantum) and then get back to the ready queue to wait for its next turn. It is a preemptive type of scheduling.

- **Shortest Job First**

The job with the shortest burst time will get the CPU first. The lesser the burst time, the sooner will the process get the CPU. It is the non-preemptive type of scheduling.

- **Shortest remaining time first**

It is the preemptive form of SJF. In this algorithm, the OS schedules the Job according to the remaining time of the execution.

- **Priority based scheduling**

In this algorithm, the priority will be assigned to each of the processes. The higher the priority, the sooner will the process get the CPU. If the priority of the two processes is same then they will be scheduled according to their arrival time.

FIRST COME FIRST SERVE

- In the "First come first serve" scheduling algorithm, the process which arrives first, gets executed first, or we can say that the process which requests the CPU first, gets the CPU allocated first.
- First Come First Serve, is just like **FIFO**(First in First out) Queue data structure, where the data element which is added to the queue first, is the one who leaves the queue first.
- This is used in Batch Systems.
- It's **easy to understand and implement** programmatically, using a Queue data structure, where a new process enters through the **tail** of the queue, and the scheduler selects process from the **head** of the queue.
- A perfect real life example of FCFS scheduling is **buying tickets at ticket counter**.

Calculating Average Waiting Time

- AWT or Average waiting time is the average of the waiting times of the processes in the queue, waiting for the scheduler to pick them for execution.
- *Lower the Average Waiting Time, better the scheduling algorithm.*

Problems with FCFS Scheduling

- It is **Non Pre-emptive** algorithm, which means the **process priority** doesn't matter.
- Not optimal Average Waiting Time.
- Resources utilization in parallel is not possible, which leads to **Convoy Effect**, and hence poor resource(CPU, I/O etc) utilization.

Convoy Effect is a situation where many processes, who need to use a resource for short time are blocked by one process holding that resource for a long time.

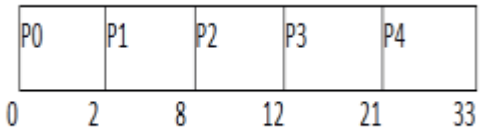
This essentially leads to poor utilization of resources and hence poor performance.

Let's take an example of The FCFS scheduling algorithm. In the Following schedule, there are 5 processes with process ID **P0, P1, P2, P3 and P4**. P0 arrives at time 0, P1 at time 1, P2 at time 2, P3 arrives at time 3 and Process P4 arrives at time 4 in the ready queue. The processes and their respective Arrival and Burst time are given in the following table.

The Turnaround time and the waiting time are calculated by using the following formula.

Turn Around Time = Completion Time - Arrival Time

Waiting Time = Turnaround time - Burst Time



Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
0	0	2	2	2	0
1	1	6	8	7	1
2	2	4	12	10	6
3	3	9	21	18	9
4	4	12	33	29	17

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4



P4 = 0-0 = 0

P3 = 3-1 = 2

PI = 11-2 = 9

P5= 17-4 = 13

P2= 21-5= 16

Average Waiting Time = 40/5 = 8

SHORTEST JOB FIRST SCHEDULING

- It works on the process with the shortest **burst time** or **duration** first.
- This is the best approach to minimize waiting time.
- This is used in Batch Systems where required CPU time is known in advance.
- It is of two types:
  - Non Pre-emptive
  - Pre-emptive
- To successfully implement it, the burst time/duration time of the processes should be known to the processor in advance, which is practically not feasible all the time.

This scheduling algorithm is optimal if all the jobs/processes are available at the same time. (either Arrival time is 0 for all, or Arrival time is same for all)

Non Pre-emptive Shortest Job First

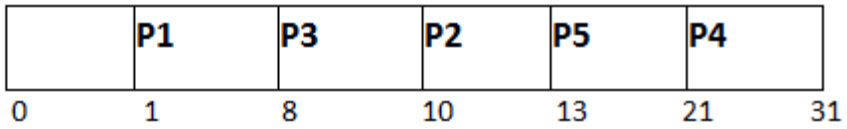
The processes are available in the ready queue for execution, with **arrival time** as 0 for all and given **burst times**.

Problem with Non Pre-emptive SJF

- If the **arrival time** for processes are different, which means all the processes are not available in the ready queue at time 0, and some jobs arrive after some time, in such situation, sometimes process with short burst time have to wait for the current process's execution to finish, because in Non Pre-emptive SJF, on arrival of a process with short duration, the existing job/process's execution is not halted/stopped to execute the short job first.
- This leads to the problem of **Starvation**, where a shorter process has to wait for a long time until the current longer process gets executed. This happens if shorter jobs keep coming, but this can be solved using the concept of **aging**.

In the following example, there are five jobs named as P1, P2, P3, P4 and P5. Their arrival time and burst time are given in the table below.

PID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	1	7	8	7	0
2	3	3	13	10	7
3	6	2	10	4	2
4	7	10	31	24	14
5	9	8	21	12	4

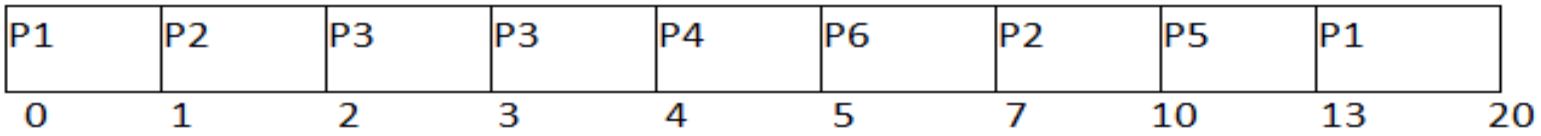


Shortest Remaining Time First (SRTF) Scheduling Algorithm (Pre-emptive Shortest Job First)

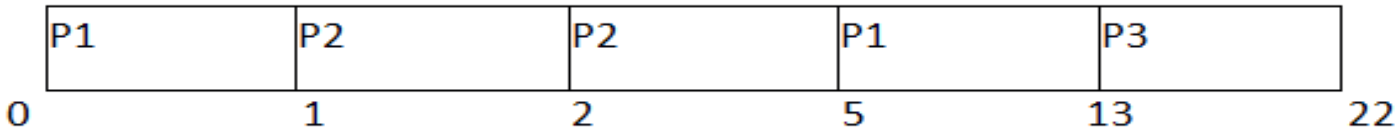
- In Preemptive Shortest Job First Scheduling, jobs are put into ready queue as they arrive, but as a process with **short burst time** arrives, the existing process is preempted or removed from execution, and the shorter job is executed first
- The Pre-emptive SJF is also known as **Shortest Remaining Time First**, because at any given point of time, the job with the shortest remaining time is executed first. At the arrival of every process, the short term scheduler schedules the process with the least remaining burst time among the list of available processes and the running process.

In the Example, there are five jobs P1, P2, P3, P4, P5 and P6. Their arrival time and burst time are given below in the table.

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time	Response Time
1	0	8	20	20	12	0
2	1	4	10	9	5	1
3	2	2	4	2	0	2
4	3	1	5	2	1	4
5	4	3	13	9	6	10
6	5	2	7	2	0	5



Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	0	9	13	13	4
2	1	4	5	4	0
3	2	9	22	20	11

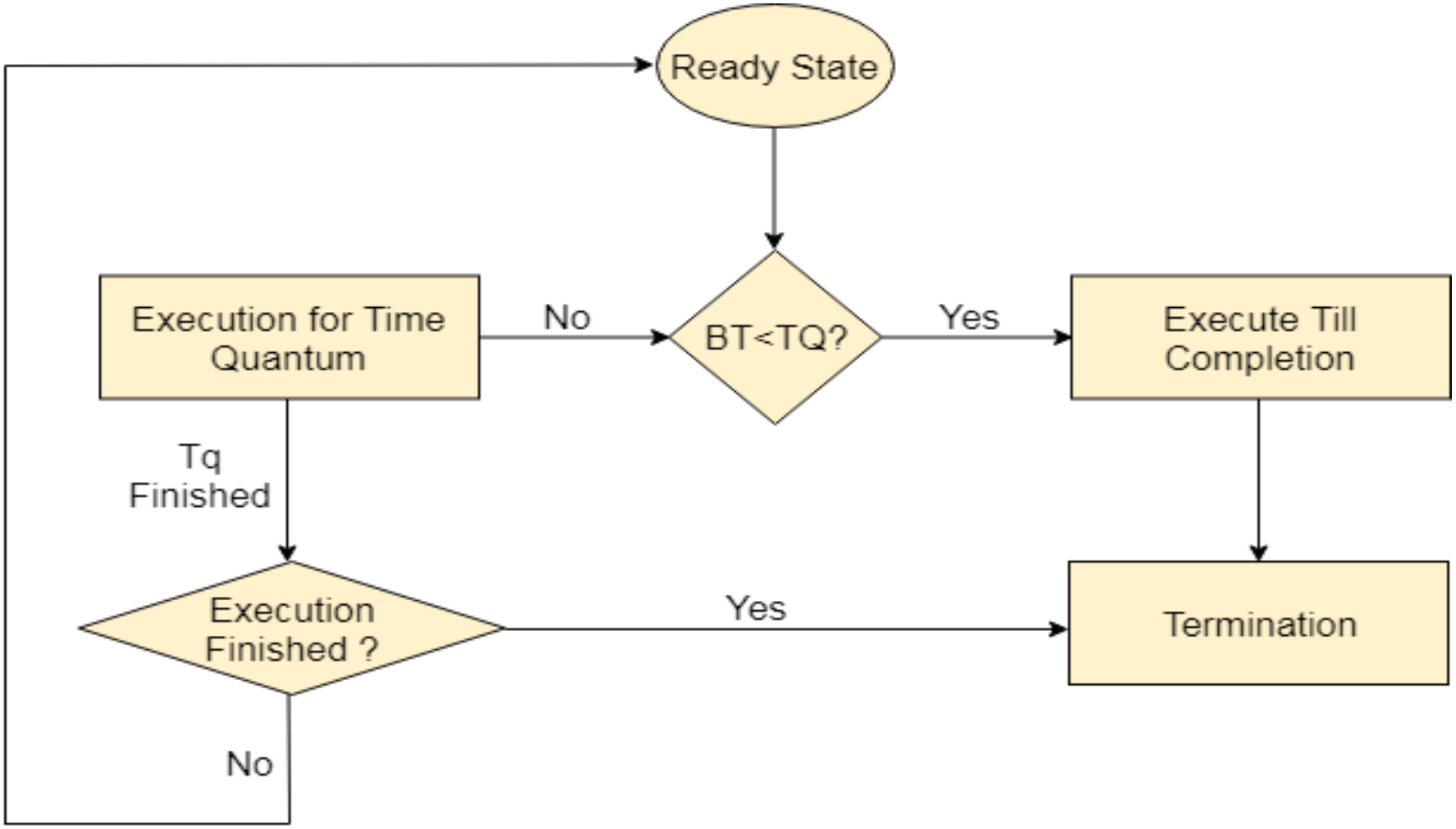


ROUND ROBIN SCHEDULING

- Round Robin(RR) scheduling algorithm is mainly designed for time-sharing systems. This algorithm is similar to FCFS scheduling, but in Round Robin(RR) scheduling, preemption is added which enables the system to switch between processes.
- A fixed time is allotted to each process, called a **quantum**, for execution.
- Once a process is executed for the given time period that process is preempted and another process executes for the given time period.
- Context switching is used to save states of preempted processes.
- This algorithm is simple and easy to implement and the most important is thing is this algorithm is starvation-free as all processes get a fair share of CPU.
- It is important to note here that the length of time quantum is generally from 10 to 100 milliseconds in length.
- Some important characteristics of the Round Robin(RR) Algorithm are as follows:

- Round Robin Scheduling algorithm resides under the category of Preemptive Algorithms.
- This algorithm is one of the oldest, easiest, and fairest algorithm.
- This Algorithm is a real-time algorithm because it responds to the event within a specific time limit.
- In this algorithm, the time slice should be the minimum that is assigned to a specific task that needs to be processed. Though it may vary for different operating systems.
- This is a hybrid model and is clock-driven in nature.
- This is a widely used scheduling method in the traditional operating system.
- Important terms
- **Completion Time** It is the time at which any process completes its execution.
- **Turn Around Time** This mainly indicates the time Difference between completion time and arrival time. The Formula to calculate the same is: **Turn Around Time = Completion Time – Arrival Time**

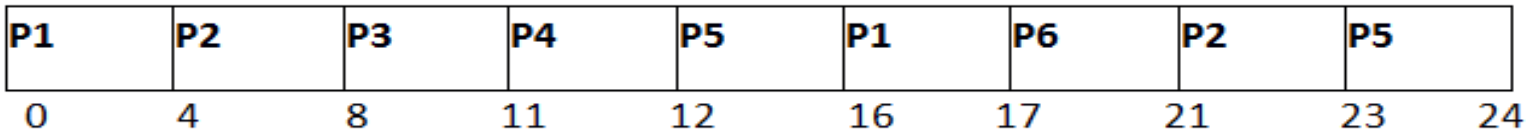
- **Waiting Time(W.T):** It Indicates the time Difference between turn around time and burst time. And is calculated as **Waiting Time = Turn Around Time – Burst Time**



In the following example, there are six processes named as P1, P2, P3, P4, P5 and P6. Their arrival time and burst time are given below in the table. The time quantum of the system is 4 units.

Process ID	Arrival Time	Burst Time
1	0	5
2	1	6
3	2	3
4	3	1
5	4	5
6	6	4

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	0	5	17	17	12
2	1	6	23	22	16
3	2	3	11	9	6
4	3	1	12	9	8
5	4	5	24	20	15
6	6	4	21	15	11



Meanwhile the execution of P1, four more processes P2, P3, P4 and P5 arrives in the ready queue. P1 has not completed yet, it needs another 1 unit of time hence it will also be added back to the ready queue.

P2	P3	P4	P5	P1
6	3	1	5	1

During the execution of P2, one more process P6 is arrived in the ready queue. Since P2 has not completed yet hence, P2 will also be added back to the ready queue with the remaining burst time 2 units.

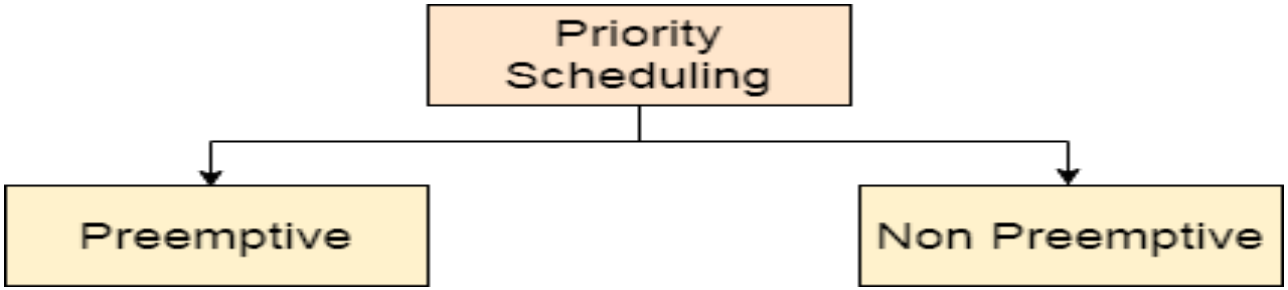
P3	P4	P5	P1	P6	P2
3	1	5	1	4	2

PRIORITY SCHEDULING

- In case of priority scheduling the priority is not always set as the inverse of the CPU burst time, rather it can be internally or externally set, but yes the scheduling is done on the basis of priority of the process where the process which is most urgent is processed first, followed by the ones with lesser priority in order.
- Processes with same priority are executed in FCFS manner.
- The priority of process, when internally defined, can be decided based on **memory requirements**, **time limits** ,**number of open files**, **ratio of I/O burst to CPU burst** etc.
- Whereas, external priorities are set based on criteria outside the operating system, like the importance of the process, funds paid for the computer resource use, market factor etc.

In some systems, the lower the number, the higher the priority. While, in the others, the higher the number, the higher will be the priority.

There are two types of priority scheduling algorithm exists. One is Preemptive priority scheduling while the other is Non Preemptive Priority scheduling.



The priority number assigned to each of the process may or may not vary. If the priority number doesn't change itself throughout the process, it is called **static** priority, while if it keeps changing itself at the regular intervals, it is called **dynamic** priority.

- **Non-Preemptive Priority Scheduling**: The Processes are scheduled according to the priority number assigned to them. Once the process gets scheduled, it will run till the completion. if a new process arrives with a higher priority than the current running process, the incoming process is put at the head of the ready queue.

- **Preemptive Priority Scheduling**: If the new process arrived at the ready queue has a higher priority than the currently running process, the CPU is preempted, which means the processing of the current process is stopped and the incoming new process with higher priority gets the CPU for its execution.

- Problem with Priority Scheduling Algorithm

- In priority scheduling algorithm, there are chances of **indefinite blocking** or **starvation**.
- A process is considered blocked when it is ready to run but has to wait for the CPU as some other process is running currently.

Non Preemptive Priority Scheduling

In the Example, there are 7 processes P1, P2, P3, P4, P5, P6 and P7. Their priorities, Arrival Time and burst time are given in the table.

Process ID	Priority	Arrival Time	Burst Time
1	2	0	3
2	6	2	5
3	3	1	4
4	5	4	2
5	7	6	9
6	4	5	4
7	10	7	10



Process Id	Priority	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time	Response Time
1	2	0	3	3	3	0	0
2	6	2	5	18	16	11	13
3	3	1	4	7	6	2	3
4	5	4	2	13	9	7	11
5	7	6	9	27	21	12	18
6	4	5	4	11	6	2	7
7	10	7	10	37	30	18	27

Preemptive Priority Scheduling

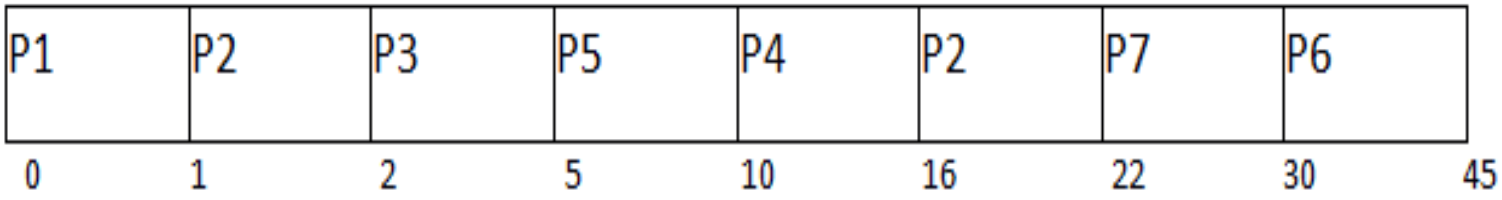
In Preemptive Priority Scheduling, at the time of arrival of a process in the ready queue, its Priority is compared with the priority of the other processes present in the ready queue as well as with the one which is being executed by the CPU at that point of time. The One with the highest priority among all the available processes will be given the CPU next.

Once all the jobs get available in the ready queue, the algorithm will behave as non-preemptive priority scheduling, which means the job scheduled will run till the completion and no preemption will be done.

Example

There are 7 processes P1, P2, P3, P4, P5, P6 and P7 given. Their respective priorities, Arrival Times and Burst times are given in the table below.

Process Id	Priority	Arrival Time	Burst Time
1	2(H)	0	1
2	6	1	7
3	3	2	3
4	5	3	6
5	4	4	5
6	10(L)	5	15
7	9	15	8



Process Id	Priority	Arrival Time	Burst Time	Completion Time	Turn around Time	Waiting Time
1	2	0	1	1	1	0
2	6	1	7	22	21	14
3	3	2	3	5	3	0
4	5	3	6	16	13	7
5	4	4	5	10	6	1
6	10	5	15	45	40	25
7	9	6	8	30	24	16

Multilevel Queue Scheduling

It may happen that processes in the ready queue can be divided into different classes where each class has its own scheduling needs. For example, a common division is a foreground (interactive) process and a background (batch) process. These two classes have different scheduling needs. For this kind of situation, Multilevel Queue Scheduling is used.

Features of Multilevel Queue (MLQ) CPU Scheduling:

- Multiple queues: In MLQ scheduling, processes are divided into multiple queues based on their priority, with each queue having a different priority level. Higher-priority processes are placed in queues with higher priority levels, while lower-priority processes are placed in queues with lower priority levels.

- Priorities assigned: Priorities are assigned to processes based on their type, characteristics, and importance. For example, interactive processes like user input/output may have a higher priority than batch processes like file backups.

- Preemption: Preemption is allowed in MLQ scheduling, which means a higher priority process can preempt a lower priority process, and the CPU is allocated to the higher priority process. This helps ensure that high-priority processes are executed in a timely manner.

- Scheduling algorithm: Different scheduling algorithms can be used for each queue, depending on the requirements of the processes in that queue. For example, Round Robin scheduling may be used for interactive processes, while First Come First Serve scheduling may be used for batch processes.

- Feedback mechanism: A feedback mechanism can be implemented to adjust the priority of a process based on its behavior over time. For example, if an interactive process has been waiting in a lower-priority queue for a long time, its priority may be increased to ensure it is executed in a timely manner.

- Efficient allocation of CPU time: MLQ scheduling ensures that processes with higher priority levels are executed in a timely manner, while still allowing lower priority processes to execute when the CPU is idle.

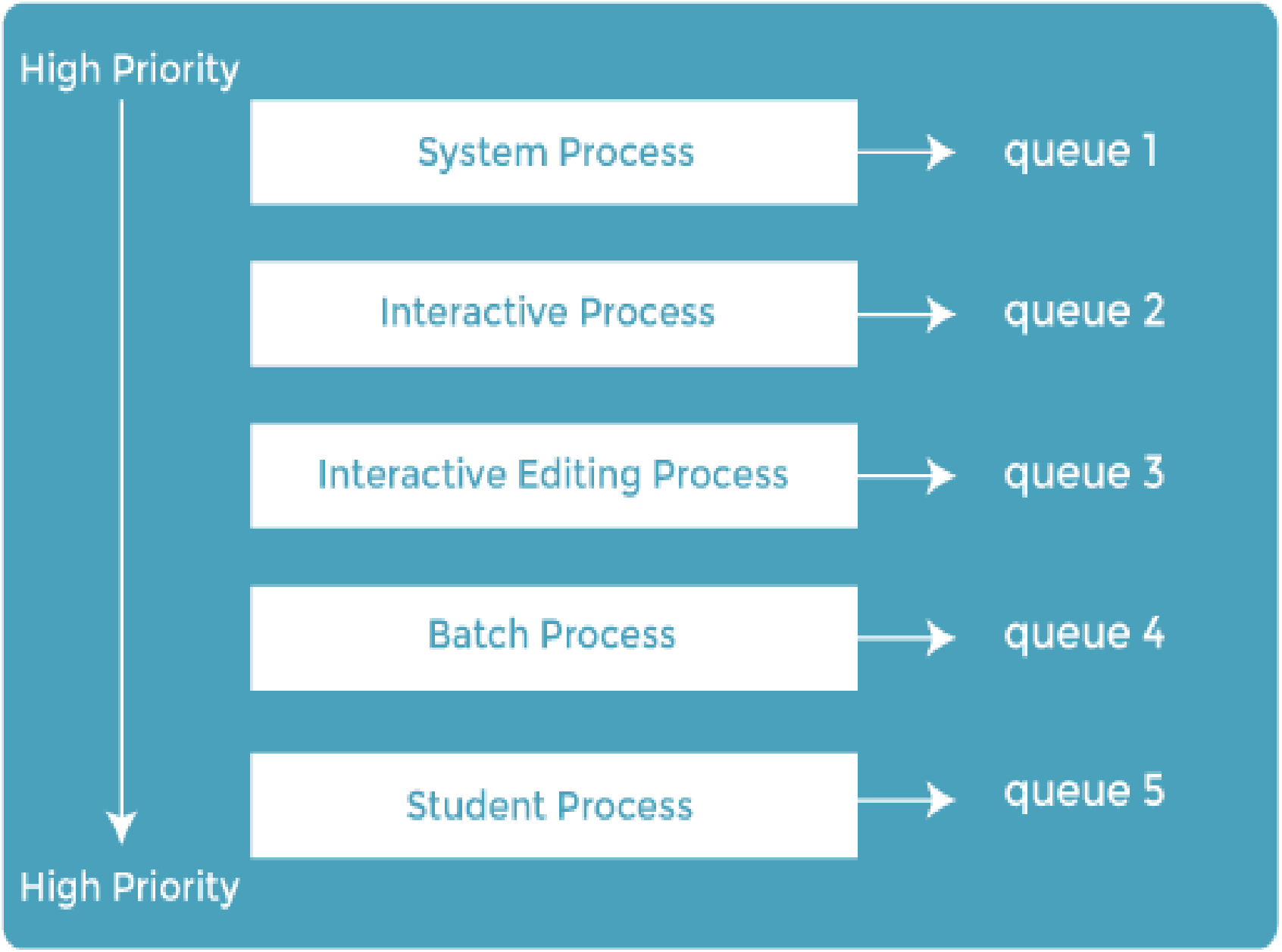
- Fairness: MLQ scheduling provides a fair allocation of CPU time to different types of processes, based on their priority and requirements.

- Customizable: MLQ scheduling can be customized to meet the specific requirements of different types of processes.

The ready queue has been partitioned into seven different queues using the multilevel queue scheduling technique. These processes are assigned to one queue based on their priority, such as memory size, process priority, or type. The method for scheduling each queue is different. Some queues are utilized for the foreground process, while others are used for the background process. The foreground queue may be scheduled using a round-robin method, and the background queue can be scheduled using an FCFS strategy.

Example

Let's take an example of a multilevel queue-scheduling algorithm with five queues to understand how this scheduling works:


High Priority
System Process
queue 1
Interactive Process
queue 2
Interactive Editing Process
queue 3
Batch Process
queue 4
Student Process
queue 5
High Priority

Every queue would have an absolute priority over the low-priority queues. No process may execute until the high-priority queues are empty. In the above instance, no other process may execute until and unless the queues for system, interactive, and editing processes are empty. If an interactive editing process enters the ready queue while a batch process is underway, the batch process will be preempted.

There are the descriptions of the processes that are used in the above example:

- System Process

The OS has its process to execute, which is referred to as the System Process.

- Interactive Process

It is a process in which the same type of interaction should occur.

- Batch Process

Batch processing is an operating system feature that collects programs and data into a batch before processing starts.

Student Process

The system process is always given the highest priority, whereas the student processes are always given the lowest.

Example Problem

Let's take an example of a multilevel queue-scheduling (MQS) algorithm that shows how the multilevel queue scheduling work. Consider the four processes listed in the table below under multilevel queue scheduling. The queue number denotes the process's queue.

Process	Arrival Time	CPU Burst Time	Queue Number
P1	0	4	1
P2	0	3	1
P3	0	8	2
P4	10	5	4

Queue 1 has a higher priority than queue 2. Round Robin is used in queue 1 (Time Quantum = 2), while FCFS is used in queue 2.



In an MLQ (Multilevel Queue) architecture, the system is divided into multiple queues or levels based on the type of processes. The MLQ architecture aims to efficiently handle different types of processes and prioritize their execution based on their characteristics.

The main components of an MLQ architecture are

System Processes − System processes refer to operating system tasks or processes that are essential for the proper functioning of the system. These processes handle critical operations such as memory management, process scheduling, I/O handling, and other system-level functions. System processes typically have the highest priority in the MLQ architecture and are executed before other types of processes.

- Interactive Processes − Interactive processes are user-oriented processes that require an immediate or near-real-time response. These processes are typically initiated by user interaction, such as keyboard input or mouse clicks. Examples of interactive processes include running a text editor, web browser, or a graphical application. Interactive processes are given priority in the MLQ architecture to ensure a smooth and responsive user experience.
- Batch Processes − Batch processes are non-interactive processes that are executed in the background without direct user interaction. These processes often involve executing a series of tasks or jobs that can be automated and executed without user intervention. Batch processes are typically resource-intensive and can include tasks such as data processing, large-scale computations, or scheduled backups. In the MLQ architecture, batch processes are allocated resources based on their priority, but they are generally given lower priority compared to interactive processes.

Different Levels of Queues

Processes are divided into many tiers of queues in multilevel queue scheduling according to their features, priority, and time limitations. The following various layers of queues can be utilized in multilevel queue scheduling:

- Foreground Queue − Interactive processes that demand quick system responses often use this queue. Usually, these procedures receive first attention
- Background Queue − This queue is used for non-interactive processes that take longer to complete. These tasks are prioritized less than foreground tasks.
- Interactive queue − The interactive queue is used for processes that demand a system response in a reasonable amount of time
- Batch Queue − This queue is used for batch processing, or processing many jobs at once. These tasks typically call for lengthy execution times and are submitted in advance.

- System Queue − This queue is used for system processes, such as device drivers and interrupt handlers, that need special permissions to complete their tasks. Usually, these procedures receive first attention.
- Real-time Queue − Real-time processes, which demand a prompt response from the system, are queued in this way. Usually, these procedures receive first attention.

Implementation of Multilevel Queue Scheduling

When using multilevel queue scheduling, the processes are split up into different queues according to their characteristics and priority, and then they are scheduled using the scheduling algorithm for each queue. The following steps are needed to implement multilevel queue scheduling −

- Sort the processes into several tiers of queues according to their attributes, such as priority, deadlines, and I/O demands.

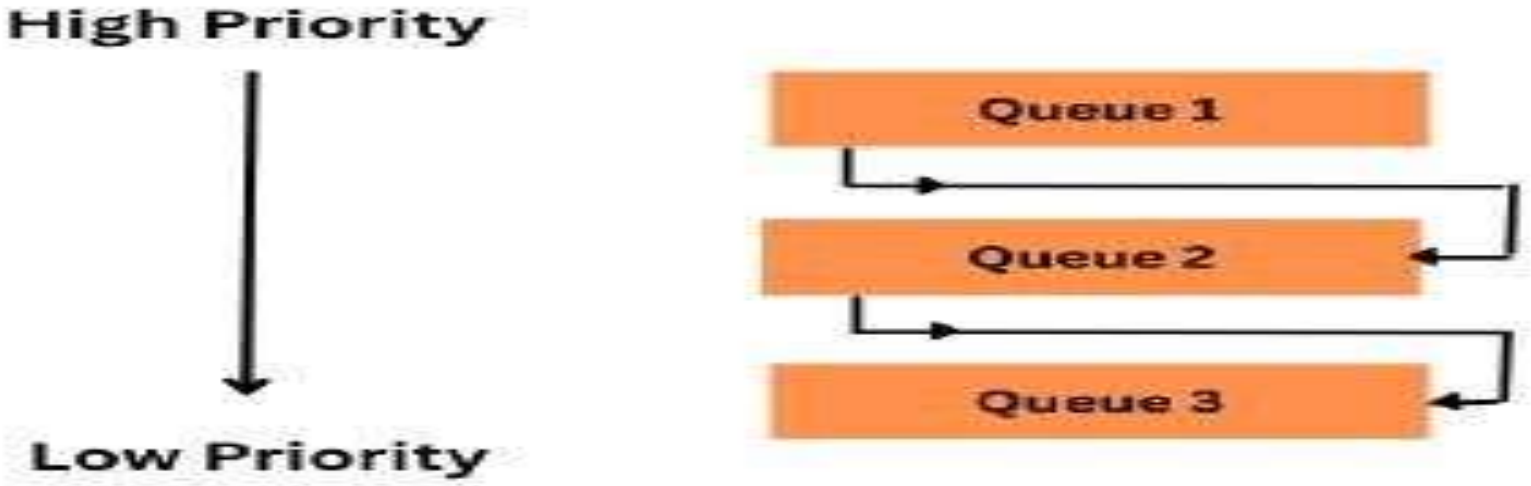
- Choose the optimal scheduling algorithm for each queue based on the kinds of processes it contains. For instance, the batch queue might employ a First-Come-First- Serve (FCFS) scheduling strategy whereas the foreground queue might use a round robin scheduling algorithm.
- Give each queue a priority based on how important the processes in it are. For instance, the batch queue might have the lowest priority while the foreground queue might have the greatest priority
- A process is positioned in the proper queue when it enters the system based on its characteristics. An interactive process, for instance, could go in the interactive queue, whereas a batch job could go in the batch queue.

- Using the appropriate scheduling method for that queue, the scheduler chooses a process from the top of the queue that has a process scheduled to be executed and prepares it for execution

- The scheduler chooses the next process to be executed from the highest priority queue that contains a process waiting to be executed whenever the process has finished its execution or is blocked on I/O.

- The procedure keeps going until every process in the system has finished running.

MLFQ Programming is an instance of CPU scheduling technique



MLFQ Programming is an instance of CPU scheduling technique that works through preserving several queues based on priority, each with a distinct time quantum. The delay period of those with greater importance queues is shorter, whereas the duration period of the ones with lower importance holds is longer.

When a fresh procedure emerges, it is assigned to the top of the importance queue. The timer for the CPU chooses the most significant operation from the most important queue and allocates the processing power to it. The procedure is permitted to continue for a set amount of time or as long as it is completed, whatever comes first. If the task finishes prior to the duration of the period running out, the processor's scheduler moves on to the next task in the queue.

Features of Multilevel Feedback Queue Scheduling (MLFQ) CPU Scheduling:

- Multiple queues: Similar to MLQ scheduling, MLFQ scheduling divides processes into multiple queues based on their priority levels. However, unlike MLQ scheduling, processes can move between queues based on their behavior and needs.

- Priorities adjusted dynamically: The priority of a process can be adjusted dynamically based on its behavior, such as how much CPU time it has used or how often it has been blocked. Higher-priority processes are given more CPU time and lower-priority processes are given less.

- Time-slicing: Each queue is assigned a time quantum or time slice, which determines how much CPU time a process in that queue is allowed to use before it is preempted and moved to a lower priority queue.

- Feedback mechanism: MLFQ scheduling uses a feedback mechanism to adjust the priority of a process based on its behavior over time. For example, if a process in a lower-priority queue uses up its time slice, it may be moved to a higher-priority queue to ensure it gets more CPU time.

- Preemption: Preemption is allowed in MLFQ scheduling, meaning that a higher-priority process can preempt a lower-priority process to ensure it gets the CPU time it needs.

Important Points

If the allotted quantum runs out beforehand the procedure is finished, it is delayed and shifted into the following queue. The process that needs execution is then chosen from the list of processes with the highest priority. This procedure is repeated as long as the procedure is finished or it attains the lowest level of the queue.

MLFQ scheduling, along with preempting procedures depending on their duration quantum, employs aging methods to guarantee that good-running procedures are ultimately given an opportunity to execute. Since its position of importance rises as it utilizes a longer period within a lower-priority queue. This guarantees that long waiting processes ultimately move into a higher-priority list and receive CPU time.

Implementation of MFQS is given below –

- When a process starts executing the operating system can insert it into any of the above three queues depending upon its priority. For example, if it is some background process, then the operating system would not like it to be given to higher priority queues such as queues 1 and 2. It will directly assign it to a lower priority queue i.e. queue 3. Let's say our current process for consideration is of significant priority so it will be given queue 1.
- In queue 1 process executes for 4 units and if it completes in these 4 units or it gives CPU for I/O operation in these 4 units then the priority of this process does not change and if it again comes in the ready queue then it again starts its execution in Queue 1.
- If a process in queue 1 does not complete in 4 units then its priority gets reduced and it is shifted to queue 2.

- Above points 2 and 3 are also true for queue 2 processes but the time quantum is 8 units. In a general case if a process does not complete in a time quantum then it is shifted to the lower priority queue.
- In the last queue, processes are scheduled in an FCFS manner.
- A process in a lower priority queue can only execute only when higher priority queues are empty.
- A process running in the lower priority queue is interrupted by a process arriving in the higher priority queue.

Example:

Consider a system that has a CPU-bound process, which requires a burst time of 40 seconds. The multilevel Feed Back Queue scheduling algorithm is used and the queue time quantum ‘2’ seconds and in each level it is incremented by ‘5’ seconds. Then how many times the process will be interrupted and in which queue the process will terminate the execution?

Solution:

- Process P needs 40 Seconds for total execution.
- At Queue 1 it is executed for 2 seconds and then interrupted and shifted to queue 2.
- At Queue 2 it is executed for 7 seconds and then interrupted and shifted to queue 3.

- At Queue 3 it is executed for 12 seconds and then interrupted and shifted to queue 4.
- At Queue 4 it is executed for 17 seconds and then interrupted and shifted to queue 5.
- At Queue 5 it executes for 2 seconds and then it completes.
- Hence the process is interrupted 4 times and completed on queue 5.